

Project Reflection: Client Server Model

Assignment: Dual-Panel Activity & Protocol Visualizer

Course: Computer Networks – Application Layer

1. AI Platforms & Models Used

Platform: Google Antigravity

Model: Gemini 3.1 Pro (High)

Rationale: The Antigravity platform with the Gemini Pro model was chosen due to its advanced agentic capabilities, particularly its proficiency in reasoning about UI layouts (Tailwind CSS) and accurate network protocol sequences. It effectively handled multi-file project generation and complex asynchronous sequence diagram logic in a single context window.

2. Implementation & Synchronization Architecture

The application is built using a pure frontend approach (HTML, CSS with Tailwind, and Vanilla JavaScript). This ensures zero backend dependencies while maintaining high performance for animations.

- **Left Panel (Activity):** Uses HTML forms to capture user inputs for Browsing (URL), Mail (To/Subject/Body), and Streaming (Quality).
- **Right Panel (Visualization):** Uses absolute positioning and CSS transitions (keyframes) to draw animated lifelines between Client, DNS Server, and App/CDN Servers.
- **Synchronization:** When an action is submitted on the Left Panel, an event listener in `app.js` immediately triggers the `startSequence()` function. This function dynamically generates an array of protocol step objects (e.g., DNS queries, TCP handshakes, HTTP GET requests) based on the user's exact input. A state machine then steps through these objects, rendering the arrows and messages on the Right Panel in perfect synchrony with the activity log on the left.

3. AI Corrections and Iterations

During development, the AI initially attempted to serve the application using a Python FastAPI backend. However, upon reviewing the assignment requirements which stated "A pure frontend (React/Vue) is allowed," I directed the AI to pivot to a pure HTML/JS/CSS implementation. This simplified the deployment process and made the visual UI theme (Cyberpunk/Neon) easier to implement using Tailwind CSS animations.

Additionally, the AI initially created a very linear protocol flow. I instructed the AI to ensure accurate multi-step protocol flows, such as the full SMTP conversation (EHLO, MAIL FROM, RCPT TO, DATA, QUIT), which it successfully implemented in the sequence generator.

4. Key Protocol Observations

DNS + HTTP (Browsing)

The browsing sequence is relatively short but involves two distinct phases: discovering the IP address via DNS (UDP), followed by establishing a stateful connection via TCP to send the HTTP GET request. The stateless nature of HTTP requires the entire document to be fetched in one go (or chunked), but the request-response pair is conceptually single-turn.

SMTP (Mail)

SMTP is significantly more conversational and stateful. Unlike HTTP's single GET request, SMTP requires a series of distinct commands (EHLO, MAIL, RCPT, DATA) where each step depends on a `250 OK` or `354 Start mail input` response from the server. It represents a strict state machine compared to the more straightforward HTTP request.

HTTP Streaming

Streaming over HTTP contrasts with standard browsing by demonstrating multiple, continuous HTTP GET requests. First, the client fetches a manifest file (`.m3u8`). Based on this manifest, it subsequently makes multiple HTTP GET requests for individual video segments (`.ts` files). This shows how modern streaming relies on consecutive, rapid HTTP transactions rather than a single long-lived proprietary streaming protocol.